LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 3
Student Name: Mohammed Almas
Roll No.: A24126552098
Date: 25-08-2026

---

1. TITLE

Implementation of JavaScript Arrays, Object-Oriented Programming and Functions

---

2. AIM

To implement and understand basic JavaScript concepts including **arrays and array methods, Object-Oriented Programming using classes, private fields and methods, and functions with parameters**.

---

3. OBJECTIVE

The objectives of this experiment are:

- To create and manipulate arrays in JavaScript.
- To access array elements using index numbers.
- To find the length of an array.
- To update array elements.
- To use push(), pop(), shift() and unshift() methods.
- To iterate through an array using a for...of loop.
- To understand classes and objects in JavaScript.
- To implement encapsulation using private fields.
- To create constructors and methods.
- To create and use functions with parameters.
- To perform operations on an array using functions..

4. THEORY

JavaScript provides several features for handling data and creating reusable programs. In this experiment, three important concepts are implemented: **Arrays, Object-Oriented Programming and Functions**.

A. Arrays

An array is a collection of multiple values stored in a single variable. Array elements are accessed using index numbers, starting from **0**.

Important Array Methods

push() – Adds an element to the end of an array.

pop() – Removes the last element from an array.

shift() – Removes the first element from an array.

unshift() – Adds an element to the beginning of an array.

B. Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming approach based on **classes and objects**.

A class acts as a blueprint for creating objects.

In the BankAccount program:

- BankAccount is the class.
- myAccount is an object.
- constructor() initializes the object.
- deposit() is a method.
- #balance is a private field.

The # symbol is used to declare a private field in a JavaScript class. It cannot be directly accessed from outside the class.

C. Functions

A function is a reusable block of code that performs a particular task.

Functions can accept parameters and can be called multiple times.

In the shopping cart program:

- viewCart() displays the items.
- addItem() adds a new item.
- removeLastItem() removes the last item.

Functions make programs easier to organize, reuse and maintain.

---

5. ALGORITHM / PROCEDURE

1. Create a JavaScript file named array.js.
2. Create an array containing three colors.
3. Display the original array.
4. Access individual elements using their index values.
5. Find the number of elements using length.
6. Update an element using its index.
7. Add an element using push().
8. Remove the last element using pop().
9. Remove the first element using shift().
10. Add an element at the beginning using unshift().
11. Display all elements using a for...of loop.
12. Create another JavaScript file named oops.js.
13. Create a BankAccount class.
14. Declare a private #balance field.
15. Create a constructor to initialize the account owner.
16. Create methods to access the balance and deposit money.
17. Create an object of the BankAccount class.
18. Deposit an amount and display the balance.
19. Create a third JavaScript file named functins.js.
20. Create a shopping cart array.
21. Create a function to display cart items.
22. Create a function to add items to the cart.

23. Create a function to remove the last item.
24. Call the functions and verify the output.

---

6. PROGRAM

array.js

```
// 1. Creating an array with initial values
const colors = ["Red", "Green", "Blue"];
console.log("Original List:", colors);

// 2. Accessing items using index numbers
console.log("First color (index 0):", colors[0]);
console.log("Second color (index 1):", colors[1]);

// 3. Finding the total number of items
console.log("Total colors in list:", colors.length);

// 4. Updating an item
colors[1] = "Yellow";
console.log("After updating index 1:", colors);

// 5. Adding a new item to the END
colors.push("Purple");
console.log("After adding to the end:", colors);

// 6. Removing the last item
const removedColor = colors.pop();

console.log("Removed color:", removedColor);
console.log("Array after removing last item:", colors);

// Removing the first element
const remove = colors.shift();
console.log("Remove:", remove);

// Adding an element at the beginning
colors.unshift("Pink");
console.log("Adding at start:", colors);

// 7. Looping through the array
console.log("\n--- Printing all colors one by one ---");

for (const color of colors) {
```

```
    console.log(`Color: ${color}`);
}
```

Oops.js

```
class BankAccount {

    // Private field
    #balance = 0;

    constructor(owner) {
        this.owner = owner;
    }

    // Getter-like method
    getBalance() {
        return this.#balance;
    }

    // Setter-like operation
    deposit(amount) {

        if (amount > 0) {

            this.#balance += amount;

            console.log(`Deposited ${amount}.`);

        } else {

            console.log("Invalid deposit amount.");

        }
    }
}

// Creating an object
const myAccount = new BankAccount("Alex");

// Depositing money
myAccount.deposit(500);

// Displaying balance
console.log(myAccount.getBalance());
```

functions.js

```
const shoppingCart = [
    "Laptop",
    "Headphones",
    "Mouse Pad"
];

// Function to print all items
function viewCart(cartArray) {

    console.log(`\nYour Cart (${cartArray.length} items)`);

    if (cartArray.length === 0) {

        console.log("Your cart is empty!");

        return;
    }

    for (let i = 0; i < cartArray.length; i++) {

        console.log(`${i + 1}. ${cartArray[i]}`);
    }
}

// Function to add a new item
function addItem(cartArray, item) {

    cartArray.push(item);

    console.log(`Added to cart: ${item}`);
}

// Function to remove the last item
function removeLastItem(cartArray) {

    if (cartArray.length > 0) {

        const removed = cartArray.pop();

        console.log(`Removed from cart: ${removed}`);

    } else {
```

```
        console.log("Nothing to remove. Cart is already empty!");
    }
}

// Running the program

viewCart(shoppingCart);

addItem(shoppingCart, "Keyboard");

viewCart(shoppingCart);

removeLastItem(shoppingCart);

viewCart(shoppingCart);
```

---

7. CODE EXPLANATION

Code	Explanation
const colors = []	Creates an array
colors[0]	Accesses the first element of the array.
colors.length	Returns the number of elements.
colors[1] = "Yellow"	Updates an array element.
push()	Adds an element at the end.
pop()	Removes the last element.
shift()	Removes the first element..
<link rel="stylesheet">	Connects the HTML file with the CSS file.

unshift()	Adds an element at the beginning.
for...of	Iterates through array elements.
class BankAccount	Creates a class named BankAccount.
#balance	Declares a private field.
constructor()	Initializes an object.
this.owner	Stores the account owner's name.
getBalance()	Returns the account balance.
deposit()	Adds money to the account.
new BankAccount()	Creates an object from the class.
function viewCart()	Defines a function to display cart items.
function addItem()	Defines a function to add an item.
function removeLastItem()	Defines a function to remove the last item.
return	Stops the function execution.
if...else	Performs conditional checking.

---

8. TEST CASES AND EXECUTION DATA

A. Array Program – array.js

Test Case	Operation	Expected Result	Actual Result	Status
TC-01	Create colors array	Red, Green, Blue displayed	Displayed correctly	**Pass**
TC-02	Access index 1	Green displayed	Green displayed	**Pass**
TC-03	Check length	3 displayed	3 displayed	**Pass**
TC-04	Use push()	Purple added	Purple added	**Pass**

B. OOP Program – oops.js

Test Case	Operation	Expected Result	Actual Result	Status
TC-01	Create BankAccount object	Object created successfully	Object created	**Pass**
TC-02	Deposit 500	500 deposited	500 deposited	**Pass**
TC-03	Get balance	Balance should be 500	Balance is 500	**Pass**
TC-04	Deposit negative amount	Invalid deposit message	Invalid message displayed	**Pass**

C. Functions Program – functions.js

Test Case	Operation	Expected Result	Actual Result	Status
TC-01	View cart	Three items displayed	Three items displayed	**Pass**
TC-02	Add Keyboard	Keyboard added	Keyboard added	**Pass**
TC-03	View updated cart	Four items displayed	Four items displayed	**Pass**
TC-04	Remove last item	Keyboard removed	Keyboard removed	**Pass**

---

9. OUTPUT

A. array.js

Output 1: Output Execution

```
PS C:\CSMB98\FS LAB\Record\week-4> node array.js
```

Output 2: Output

```
PS C:\CSMB98\FS LAB\Record\week-4> node array.js
Original List: [ 'Red', 'Green', 'Blue' ]
First color (index 0): Red
Second color (index 1): Green
Total colors in list: 3
After updating index 1: [ 'Red', 'Yellow', 'Blue' ]
After adding to the end (.push): [ 'Red', 'Yellow', 'Blue', 'Purple' ]
Removed color: Purple
Array after removing last item (.pop): [ 'Red', 'Yellow', 'Blue' ]
Remove: Red
adding at start [ 'Pink', 'Yellow', 'Blue' ]

--- Printing all colors one by one ---
Color: Pink
Color: Yellow
Color: Blue
PS C:\CSMB98\FS LAB\Record\week-4>
```

B. oops.js

Output 1: Execution

```
PS C:\CSMB98\FS LAB\Record\week-4> node oops.js
```

Output 2: Output

```
PS C:\CSMB98\FS LAB\Record\week-4> node oops.js
Deposited 500.
500
PS C:\CSMB98\FS LAB\Record\week-4>
```

C. functions.js

Output 1: Execution

```
PS C:\CSMB98\FS LAB\Record\week-4> node script.js
```

Output 2: Output

```
PS C:\CSMB98\FS LAB\Record\week-4> node script.js

 Your Cart (3 items)
1. Laptop
2. Headphones
3. Mouse Pad
Added to cart: Keyboard

 Your Cart (4 items)
1. Laptop
2. Headphones
3. Mouse Pad
4. Keyboard
Removed from cart: Keyboard

 Your Cart (3 items)
1. Laptop
2. Headphones
3. Mouse Pad
PS C:\CSMB98\FS LAB\Record\week-4>
```

---

10. RESULT

Thus, the JavaScript programs were successfully implemented and executed.

The **array program** demonstrated array creation, indexing, updating, length calculation and array methods such as push(), pop(), shift() and unshift().

The **OOP program** demonstrated classes, objects, constructors, methods and private fields using JavaScript.

The **functions program** demonstrated reusable functions with parameters and array manipulation through a simple shopping cart application.

All three programs were successfully tested and the expected outputs were obtained.
